Exploiting ASP.NET TemplateParser — Part II: SharePoint (CVE-2023-33160)

Exploiting ASP.NET TemplateParser — Part II: SharePoint (CVE-2023-33160) - Markus Wulftange, Code White.

- Published: date not stated
- Original: <https://code-white.com/blog/exploiting-asp.net-templateparser-part-2/>
- Preserved from: https://code-white.com/blog/exploiting-asp.net-templateparser-part-2/ (manual-import) on 2026-08-28
- Licence: unknown

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Exploiting ASP.NET TemplateParser — Part II: SharePoint (CVE-2023-33160)

In Part I, we dug into the internals of the ASP.NET `TemplateParser` and elaborated its capabilities in respect to exploitation.

In this part, we will look into whether and how this can also be exploited to gain Remote Code Execution. While this research was originally focussed on the `TemplateParser`, the newly discovered technique was also applicable to SharePoint on-premises and SharePoint Online. So we'll elaborate on how SharePoint protects against the use of malicious code and will present a novel trick that allowed to bypass these security measures (CVE-2023-33160).

Introduction

SharePoint users are allowed to upload and create custom `.aspx` pages:

>

A fundamental assumption of the Windows SharePoint Services technology is that "untrusted users" can upload and create ASPX pages within the system on which Windows SharePoint Services is running. These users should be prevented from adding server-side code within ASPX pages, but there should be a list of approved controls that those untrusted users can use. One way to provide these controls is to create a Safe Controls list.

— https://learn.microsoft.com/en-us/previous-versions/office/developer/sharepoint-2007/ms581321(v=office.12))

For that, there is the `SPPageParserFilter` class in SharePoint that extends the `PageParserFilter` class of ASP.NET to validate the server control type specified in the source code against this Safe Controls list. This validation happens already during the tokenization step in the `TemplateParser.ProcessBeginTag(Match, string)` method:

```
// If we have a control type filter, make sure the child control is allowed
if (_pageParserFilter != null) {
    Debug.Assert(childType != null);

    if (!_pageParserFilter.AllowControlInternal(childType, subBuilder)) {
        ProcessError(SR.GetString(SR.Control_type_not_allowed, childType.FullName));
        return true;
    }
}
```

In this code excerpt the `childType` variable represents the `Type` of the specified server control. The inner workings of SharePoint's `SPPageParserFilter` were previously elaborated by Soroush Dalili in *A Security Review of SharePoint Site Pages* as well as in *Room for Escape: Scribbling Outside the Lines of Template Security* by Oleksandr Mirosh and Alvaro Muñoz. We recommend reading their work first to be on the same page.

TemplateParser Challenges and Ideas

As detailed in Part I, the parsing step of the `TemplateParser` has the following policy:

-

`Type` can only be controlled using custom server controls and the `@ Register` directive:

```
<%@ Register
    TagPrefix="MyPrefix"
    Assembly="MyAssembly"
    Namespace="MyNamespace"
%>
<MyPrefix:MyTypeName
    runat="server"
/>
```

-

Server control tag names can only be `[\w:.]+`

-

Server control type resolution is done using `Assembly.Load("MyAssembly").GetType("MyNamespace" + "." + "MyTypeName")`

-

Property types are derived from the specified server control type by reflection

Let's look into the challenges one by one.

Controlling The Property Type

>

Property types are derived from the specified server control type by reflection

Idea: Use a generic type where the type argument defines the property's type.

Consider the generic class `ExpandedWrapper<TExpandedElement>` where the generic type parameter `TExpandedElement` determines the type of the `ExpandedElement` property:

```
typeof(System.Data.Services.Internal.ExpandedWrapper<DateTime>)
    .GetProperty("ExpandedElement").PropertyType == typeof(DateTime)
```

This also means, assigning a value to the `ExpandedElement` property of a `ExpandedWrapper<DateTime>` instance would result in the `DateTimeConverter.ConvertFromInvariantString(string)` method being called.

But how can we create an instance of `ExpandedWrapper<DateTime>`?

Controlling The Custom Server Control Type

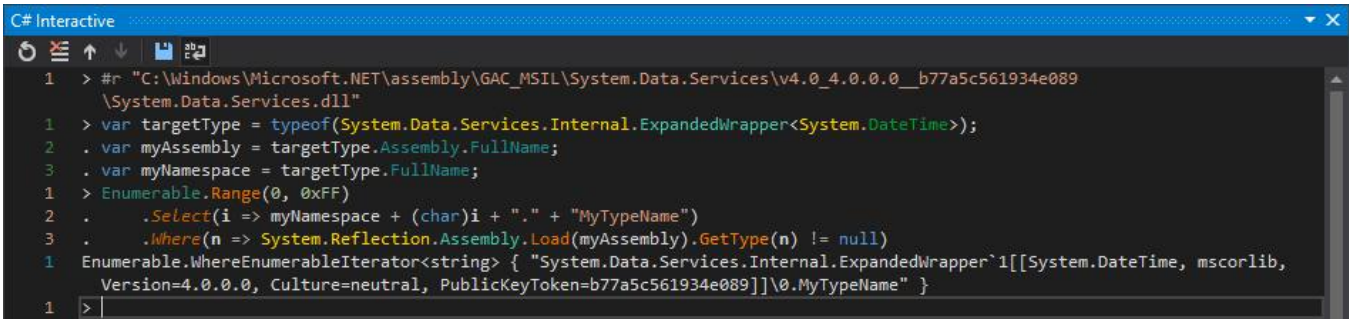
>

Server control type resolution is done using `Assembly.Load("MyAssembly").GetType("MyNamespace" + "." + "MyTypeName")`

Idea: Maybe we could trick .NET into ignoring the appendix, similar to what we have observed in *By passing .NET Serialization Binders*.

What if we could specify the targeted type in the `Namespace` entirely and somehow hide the appended `"." + "MyTypeName"` portion?

Let's make a quick test and insert an arbitrary character right after the namespace and see whether the type can still be resolved:



Surprisingly, a null character `\0` appears to terminate the parsing of the type name and the appendix `.MyTypeName` gets ignored! What year is it, again!?

>

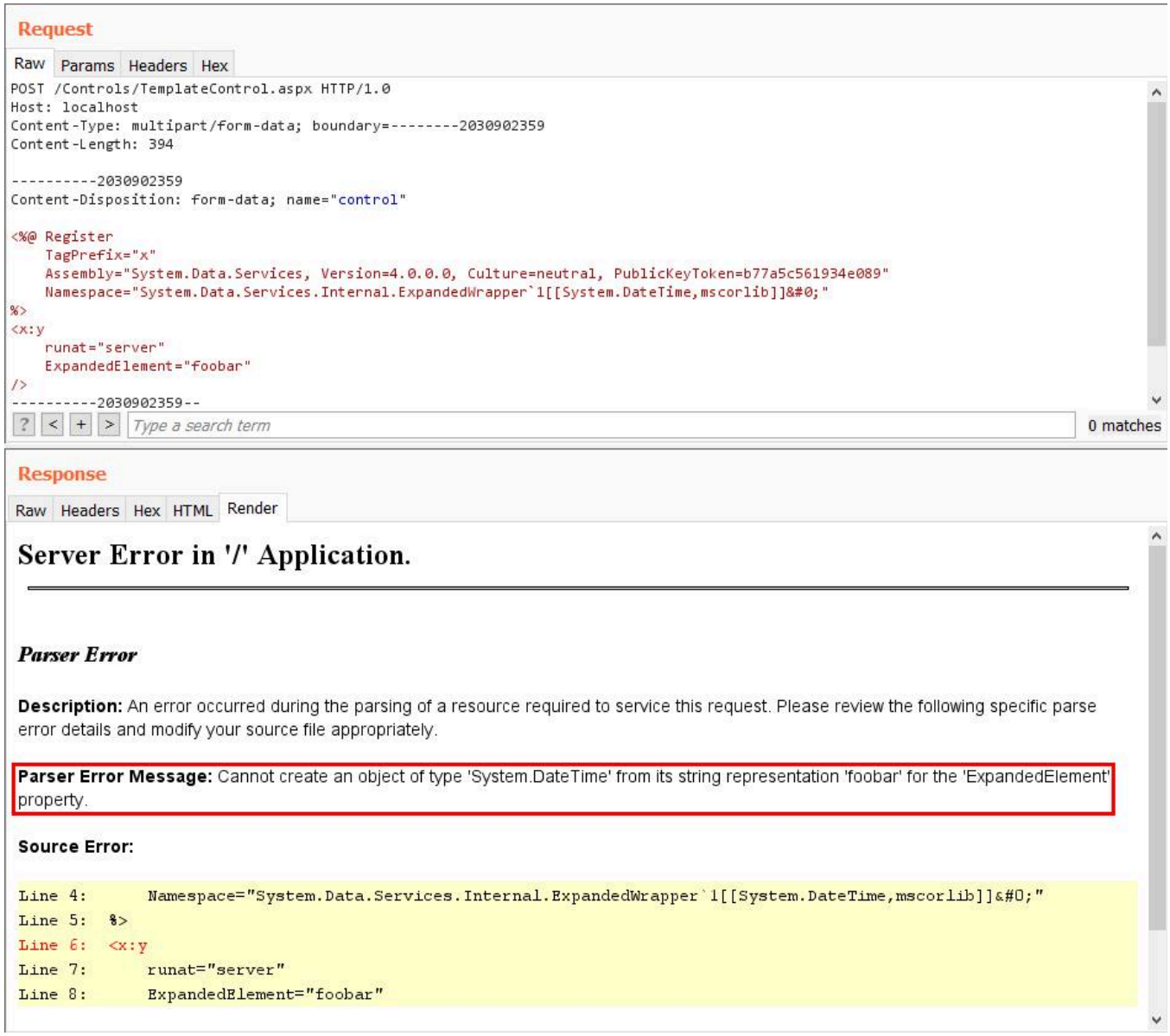
Server control tag names can only be `[\w:.]+`

Check. As the appended type name gets ignored it can be chosen arbitrarily.

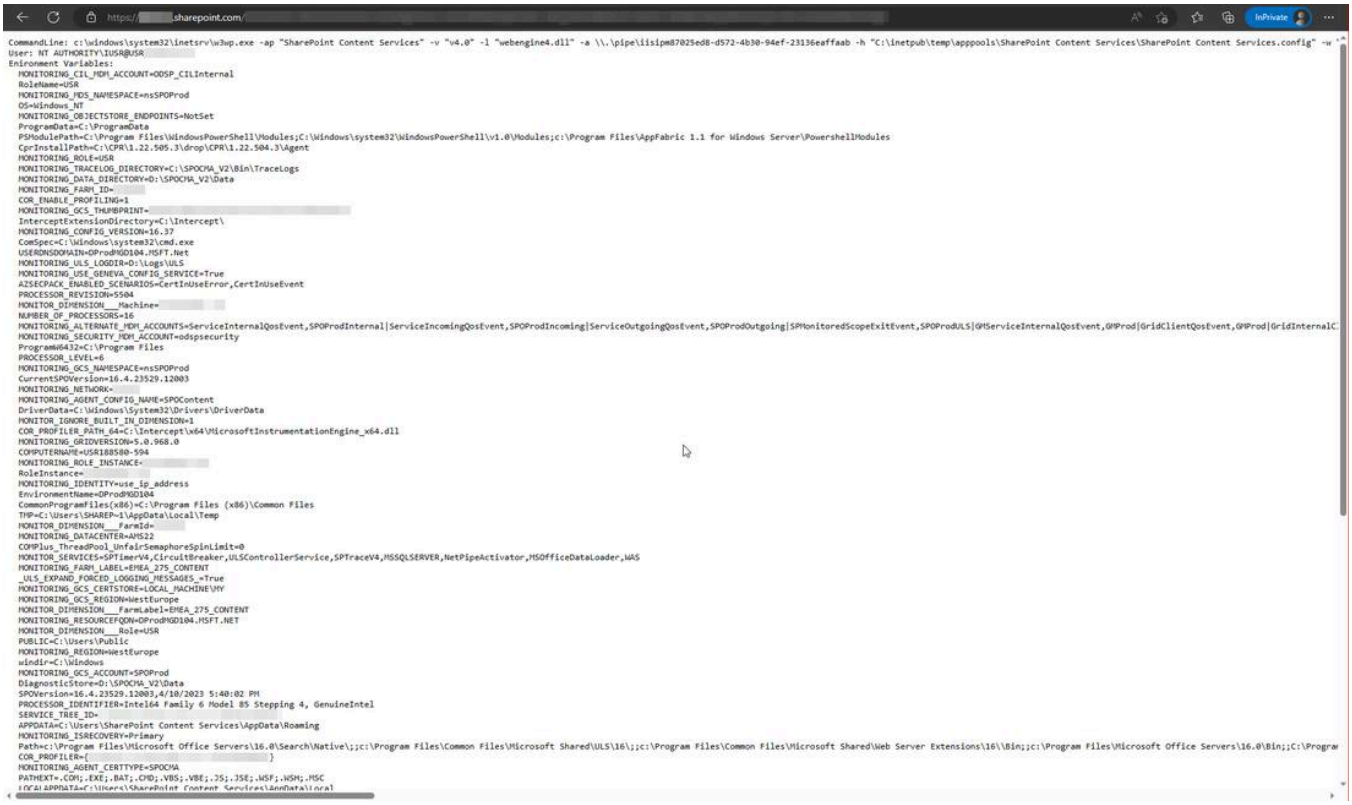
And because a null character can be appended to the `Namespace` attribute of the `@ Register` directive, we can specify a custom server control of type `ExpandedWrapper<DateTime>` and set its `ExpandedElement` property:

```
<%@ Register
   TagPrefix="x"
   Assembly="System.Data.Services, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089"
   Namespace="System.Data.Services.Internal.ExpandedWrapper`1[[System.DateTime,mscorlib]]\0"
%>
<x:y
   runat="server"
   ExpandedElement="foobar"
/>
```

Great! So this works — at least out of the context of SharePoint's `SPPageParserFilter`:



The remaining challenge was to find a generic type that passed the `SPPageParserFilter`. As well as a targeted type that provides an interesting static `Parse(string)` method or has associated an interesting `TypeConverter` that gets called on property setting. And there sure was at least one in SharePoint:



We won't publish the exploit here and leave it as an exercise to the reader.

The Patch

Microsoft fixed this vulnerability (CVE-2023-33160) in the July 2023 security updates for SharePoint by also checking the type arguments of a generic type. Here is the diff of the decompiled `SPageParserFilter` class files of the assembly file versions 16.0.16130.20548 (June 2023) and 16.0.16130.20640 (July 2023):

```
diff --git a/Microsoft.SharePoint/Microsoft/SharePoint/ApplicationRuntime/SPPageParserFilter.cs b/Microsoft.SharePoi
index d1ee368..7f893c5 100644
--- a/Microsoft.SharePoint/Microsoft/SharePoint/ApplicationRuntime/SPPageParserFilter.cs
+++ b/Microsoft.SharePoint/Microsoft/SharePoint/ApplicationRuntime/SPPageParserFilter.cs
@@ -146,9 +146,32 @@ namespace Microsoft.SharePoint.ApplicationRuntime
                         {
                             flag = false;
                         }
-                        else if ((this.IsTypeOrSubclass(typeof(Control), controlType) || !this._safeModeDefaults.Co
+                        else if ((this.IsTypeOrSubclass(typeof(Control), controlType) || !this._safeModeDefaults.Co
                         {
                             flag = this._safeControls.IsSafeControl(this._isAppWeb, controlType, out text);
+                            if (flag)
+                            {
+                                foreach (Type type in controlType.GetGenericArguments())
+                                {
+                                    try
+                                    {
+                                        flag = this.AllowControl(type, childBuilder);
+                                        if (!flag)
+                                        {
+                                            break;
+                                        }
+                                    }
+                                    catch
+                                    {
+                                        ULS.SendTraceTag(539066894U, ULSCat.msoulscat_WSS_Runtime, UL
+                                        {
+                                            type,
+                                            controlType
+                                        });
+                                        throw;
+                                    }
+                                }
+                            }
                             if (!flag && this.IsTypeOrSubclass(typeof(UserControl), controlType) && this._allowed
                             {
                                 if (this._allowUserControlTypes == null)
@@ -182,7 +205,7 @@ namespace Microsoft.SharePoint.ApplicationRuntime
                                 return false;
                             }
                             string text2;
-                            if (!this._pageParserSettings.AllowUnsafeControls && !this._safeControls.IsSafeContro
+                            if ((!this._pageParserSettings.AllowUnsafeControls || AllowUnsafeControlsOverride.A
```

```
                                    {
                                        ULS.SendTraceTag(2744449U, ULSCat.msoulscat_WSS_Runtime, ULSTraceLevel.H
                                        {
@@ -226,7 +249,7 @@ namespace Microsoft.SharePoint.ApplicationRuntime
            {
                this._IsMasterPage = this.IsTypeOrSubclass(typeof(MasterPage), baseType);
                string text = null;
-               bool flag = this.Exclusion || this._pageParserSettings.AllowUnsafeControls || this._safeControls.IsSaf
+               bool flag = this.Exclusion || (this._pageParserSettings.AllowUnsafeControls && !AllowUnsafeControlsO
                if (!flag && text != null)
                {
                    throw new SafeControls.UnsafeControlException(SPResource.GetString("UnsafeBaseTypePageP
@@ -253,7 +276,7 @@ namespace Microsoft.SharePoint.ApplicationRuntime
                    case VirtualReferenceType.Page:
                        return !SPRequestModule.IsExcludedPath(referenceVirtualPath, false);
                    case VirtualReferenceType.UserControl:
-                       flag = (this._pageParserSettings.AllowUnsafeControls || this._safeControls.IsSafeControl(th
+                       flag = ((this._pageParserSettings.AllowUnsafeControls && !AllowUnsafeControlsOverride.Al
                        if (flag)
                        {
                            if (this._allowedUserControlVirtualPaths == null)
```

Note that this patch only applies to SharePoint and not to ASP.NET in general. So if you have control over the code to be parsed by `TemplateParser`, you still can achieve Remote Code Execution.

Conclusion

The `TemplateParser` of ASP.NET holds unexpected capabilities, which can lead to Remote Code Execution during the parsing process.

Special thanks are due to the null character truncation that enables the use of generic types. This truncation also applies to assembly and type loading in general, which may allow bypassing security protections such as serialization binders (see also *Bypassing .NET Serialization Binders*).

In fact, it appears that even currently supported .NET runtimes are still affected:

Runtime	Assembly Name Truncation	Type Name Truncation
.NET Framework 4.8.1		
.NET 6		
.NET 7		
.NET 8		

The vulnerabilities in Sitecore (CVE-2023-35813) and SharePoint (CVE-2023-33160) demonstrate that vulnerabilities utilizing the ASP.NET `TemplateParser` are not just theoretical but have an actual impact on popular real world applications. And due to the fact that assembly and type name truncation still works in current .NET releases, it is likely that other vulnerable software is out there waiting to be found.